

1. Introduction (הקדמה)

רובוטים הפועלים בסביבה דינמית חייבים להבין ולהגיב לרצף של מצבים ופעולות לאורך זמן – החלטותיהם לא נקבעות על סמך תמונה יחידה, אלא מתוך מספר רגעים רציפים. הבעיה היא שבעבודה עם קלט חזותי (תמונות), בכל פריים יש כמות עצומה של פרטים (תאורה, רקע, זוויות צילום, רעש וכו'), וקשה ללמוד ישירות מתמונה גולמית. שינויים אקראיים כגון שינויי תאורה או הסתרות חלקיות עלולים להשפיע על מראה התמונה, ולכן יש צורך בייצוג חזותי יציב שמדגיש את המידע הרלוונטי למשימה ולא רגיש לרעש. בנוסף, על האלגוריתם לאזן בין דיוק לבין מהירות – נדרש פתרון שיהיה מדויק בזיהוי הפעולות, אך גם יעיל כדי שניתן יהיה להריץ ניסויים בזמן סביר ללא עומס חישובי כבד.

בפרויקט זה אני מתמקדת בלמידת רצפים חזותיים ברובוטיקה, כלומר ביכולת של מודל לחזות או לסווג את הפעולה/המצב הבא של הרובוט על סמך רצף תמונות ונתוני חיישנים. המוטיבציה לפרויקט נובעת מהצורך להתמודד עם רצפים ארוכים של מידע חזותי בזמן אמת בצורה יעילה. שילוב של שיטות למידה עמוקה חדשות מאפשר להתמודד עם אתגר זה: אני אשתמש במקודד חזותי OpenCLIP (מודל ראיה מאומן מראש) כדי להמיר כל תמונה לווקטור תכונות קומפקטי, ולאחר מכן להזין את הרצף של וקטורי התכונות הללו לתוך מודל רצף מתקדם בשם Mamba, שהוא מודל מבוסס State-Space יעיל ללמידת רצפים ארוכים. מטרת הפרויקט היא להראות ששילוב זה יאפשר לרובוט ללמוד את הדינמיקה לאורך זמן מתוך הנתונים החזותיים, ובכך לבצע חיזוי מדויק של פעולות עתידיות או הצלחת המשימה לאורך ציר הזמן.

2. System Overview (תיאור כללי של המערכת)

המערכת בנויה כ-Pipeline בשלבים, המפרידים בין עיבוד המידע החזותי לבין למידת הרצף בזמן. תחילה, קלט המערכת הוא רצף של תמונות (וידאו/פריימים), יחד עם נתוני הרובוט התואמים לכל פריים (למשל מצב גריפר, מצב מפרקים, מיקום כלי וכדומה).

את התמונות אני מזינה למקודד החזותי OpenCLIP המאומן מראש, אשר עבור כל תמונה מחזיר וקטור embedding קבוע בגודל 512 – ייצוג מספרי קומפקטי שמרכז את התוכן החזותי המרכזי של התמונה.

את סדרת הוקטורים (ה-embeddings) אני מסנכרנת עם נתוני הרובוט כך שלכל embedding מוצמד המצב/הפעולה של הרובוט באותו רגע. הרצף המשולב מוזן לאחר מכן למודל Mamba, שנועד ללמוד את התלות הטמפורלית בין מה שהרובוט "ראה" לבין מה שהוא עשה.

הפלט של Mamba יכול להיות פעולה חזויה (action), חיזוי הצלחה של המשימה, או ייצוג מצב עתידי – בהתאם למשימה המוגדרת באימון.

לסיכום, הפונקציות המרכזיות הן:

- קידוד תמונה-embedding באמצעות OpenCLIP.
- שילוב embedding עם מצב הרובוט בזמן אמת.
- הזנה למודל רצף Mamba.
- הפקת פלט (חיזוי פעולה/מצב/הצלחה).

ההפרדה בין השלבים מקלה על בדיקות וניסויים – כל שלב ניתן לשיפור או החלפה בנפרד. כך, המבנה המודולרי מאפשר לי גם להתמקד בכל תת-בעיה בנפרד וגם לשלב את הכול לפתרון שלם, יעיל וניתן להרחבה.

3. הנחות והחלטות עיצוב

3.1. הנחות מרכזיות:

3.1.1. ייצוג ביניים חזותי: נניח שנדרש שלב המרה של תמונה ל-תכונות מופשטות (embedding) לפני לימוד התלות בזמן. הנחה זו מבוססת על כך שקלט ויזואלי גולמי מכיל הרבה פרטים לא רלוונטיים או מבלבלים, וייצוג ויזואלי מקוץ מאפשר למודל הרצף להתמקד בעיקר -without being overwhelmed by pixel-level noise. לכן נבחר מקודד OpenCLIP מאומן מראש, המספק בדיוק יכולת זו – הוא ממפה כל תמונה לווקטור של 512 מספרים המייצג את התוכן המשמעותי בה בצורה יציבה וקבועה בגודל. נניח ש-OpenCLIP, שאומן על מיליוני תמונות וטקסטים, יפיק תכונות חזקות ועמידות לשינויים (למשל שינוי זווית או תאורה), וכך יחסוך את הצורך בלאמן רשת נפרדת מאפס על הדאטה.

3.1.2. שמירת רצף ארוך בזיכרון: בהיבט הזמן, ההנחה היא שעל מנת לחזות או ללמוד מרצף ארוך של מצבים, יש להשתמש במודל בעל זיכרון ארוך טווח ויעילות חישובית גבוהה. מודל ה-Mamba נבחר בהנחה שהוא יכול לשמור מידע משלבים קודמים ברצף לאורך זמן ממושך, טוב לפחות כמו Transformer מודרני, אך באופן יעיל יותר חישובית. Mamba הוא מודל מסוג State Space Model רציף, שמבנהו מאפשר מורכבות חישובית ליניארית באורך הרצף ($O(T)$), בניגוד למודלי Transformer שבהם המורכבות לרוב ריבועית ($O(T^2)$) עקב מנגנון ה-attention. ההנחה היא שיתרון זה חיוני כיוון שברצפים ארוכים מאוד (עשרות ומאות פריימים), Transformer סטנדרטי יהיה כבד מדי בזמן ובזיכרון, בעוד Mamba תוכנן במיוחד להתמודד עם רצפים כאלה בצורה קלה יותר.

3.1.3. עיבוד רב-תחומי (Multimodal): החלטת עיצוב נוספת הייתה לשלב גם את נתוני החיישנים/מצבי הרובוט יחד עם הידע החזותי. נניח שהמודל יוכל להפיק תועלת משילוב הנתונים – למשל, הידיעה מה מצב הגריפר או המפרקים יכולה לסייע לפרש נכון את התמונה. לפיכך, בעיצוב של הנתונים הנוספים משולבים ישירות בקלט למודל הרצף (עבור כל צעד זמן, ה-embedding החזותי מאוחד עם הווקטור המתאר את מצב הרובוט באותו צעד).

3.2 החלטות Design עיקריות:

3.2.1 שימוש במודל מאומן מראש לראייה: בחרתי להשתמש ב-OpenCLIP כ-Encoder חזותי מוכן במקום לאמן רשת קונבולוציות (CNN) חדשה מאפס. החלטה זו התקבלה משום שמודל CLIP כבר למד ייצוג כללי לתמונות, מה שמאפשר לקבל embedding לכל תמונה ישר "מהקופסה". כך הרווחנו גם יציבות וגם יעילות: OpenCLIP נותן לכל תמונה ייצוג וקטורי עקבי שניתן להשוות בין פריימים שונים או בין דוגמאות שונות. ללא מקודד כזה, היה חייב להוסיף רשת עמוקה אחרת לפני מודל הרצף (כמו שב-RNN/LSTM היה נדרש CNN להמרת תמונה לווקטור), מה שהיה מסרב ומייקר את תהליך האימון.

3.2.2 בחירת מודל רצף - Mamba לעומת אלטרנטיבות: לאחר בחינת אפשרויות (RNN/LSTM, Transformer, וכד'), הוחלט על Mamba כמודל הרצף. הסיבות העיקריות:

3.2.2.1 RNN/LSTM אמנם יודעים ללמוד רצפים ויעילים בזמן ריצה, אך הם סובלים מטווח זיכרון מוגבל – קשה להם לשמור מידע לאורך מאוד צעדים עקב בעיות שכחת גרדיאנט. בנוסף, הם לא מטפלים ישירות במידע חזותי ודורשים תוספת Encoder לתמונות.

3.2.2.2 Transformer מצטיינים בלמידת תלויות ארוכות טווח וייצוג עשיר, אך עבור רצפים ארוכים מאוד הם נעשים כבדים

מאוד מבחינת חישוב וזיכרון. במקרה שלי, רצף של מאות פריימים עם וקטור של 512 תכונות בכל פריים – עיבוד כזה ב-Transformer עלול להיות איטי ולא פרקטי בסביבת רובוטיקה בזמן אמת.

3.2.2.3

מודל **Mamba** לעומת זאת, מציג פתרון אופטימלי עבורי: הוא גם תומך ברצפים ארוכים (בעל זיכרון מתמשך, בדומה ל-Transformer) וגם יעיל בצורה יוצאת דופן (סיבוכיות ליניארית). למעשה, מאמרים עדכניים הראו ש-Mamba משיג ביצועים שווים ערך ל-Transformer במשימות רצף, ואף דווח כי חיזויי התנועה באמצעות Mamba היו יציבים וחלקים יותר (פחות "רעש" ורטט) בהשוואה למודלים אחרים – מאפיין קריטי ברובוטיקה, שם תנועות חלקות ויציבות הן יעד חשוב.

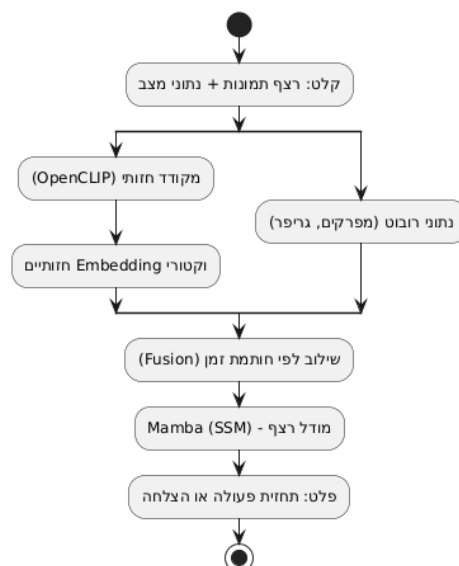
3.2.3

שילוב OpenCLIP + Mamba: ההחלטה המרכזית של התכנון היא לשלב את שני הרכיבים הנ"ל לכדי מערכת אחת. בטבלה שהוצגה במצגת ניתן היה לראות, ששילוב זה הוא היחיד שסימן ✓ בכל הקטגוריות החשובות – יכולת עבודה עם תמונות, למידה לאורך זמן, יעילות ברצפים ארוכים, ושילוב נתוני רובוט. OpenCLIP מספק ייצוג חזותי לכל פריים באופן עקבי, ו-Mamba מאחד את המידע לאורך זמן ומסוגל לשלב לתוכו גם את משתני המצב של הרובוט. כך מתקבל פתרון מודולרי ויעיל שביכולתו להתמודד עם רצפים ארוכים, להפיק תובנות מכל הפריימים, וגם לשקלל מידע נוסף (חיישנים) במידת הצורך. הפרדת התפקידים הזו הייתה החלטה מודעת: כל רכיב עושה דבר אחד ועושה אותו טוב – OpenCLIP מתמקד ב"מה רואים ברגע נתון", ו-Mamba מתמקד ב"מה המשמעות של הרצף לאורך זמן".

4. תרשים ארכיטקטורה

להלן תרשים ארכיטקטורה כללי המתאר כיצד רכיבי המערכת מתחברים יחדיו:

Robot Perception Pipeline (OpenCLIP + Mamba)



בתרשים רואים את הזרימה מן הקלט עד לפלט. בשלב הראשון, התמונות עוברות דרך OpenCLIP שיוצר עבור כל תמונה וקטור מספרי של תכונות. במקביל, נתוני הרובוט לכל תמונה משולבים (באמצעות סנכרון על פי חותמות זמן) עם הוקטורים הללו. שני הזרמים מתכנסים למודל Mamba, שלומד את הקשרים הטמפורליים ברצף המשולב. הפלט יכול

להיות רצף של תחזיות (למשל פעולה צפויה בכל צעד) או החלטה מסכמת (למשל האם הרצף מוביל להצלחת המשימה). באופן זה, הארכיטקטורה מיישמת את עיקרון הפרדת האחריות: המודול הוויזואלי מטפל ב-מה רואים בכל רגע, והמודול הטמפורלי מטפל ב-מה קורה לאורך הרצף כולו.

5. ממשקים ו-API

5.1. פורמט קלט ופלט: המערכת צפויה לקבל כקלט דאטה של רצפי רובוט – למעשה trajectory הכולל גם וידאו (רצף תמונות) וגם נתוני תנועה/חיישנים תואמים. בפועל, מידע זה יכול להגיע בקבצי וידאו+לוג, או כספרייה של תמונות (ממוספרות לפי זמן) וקובץ CSV/JSON תואם עם נתוני הרובוט בכל תמונה. ממשק הטעינה יקבל נתיב לתיקית נתונים של ניסוי אחד, או רשימת נתיבים למספר ניסויים (לצרכי אימון במקביל). עבור כל רצף נכנס, פונקציית הטעינה תפיק מבנה נתונים הכולל: רשימת תמונות (לאחר המרה למערך טנזור PyTorch, גודל כל תמונה אחרי עיבוד מקדים למשל 224X224 פיקסלים), ומטריצה של נתוני הרובוט (גודל $N \times d$, כאשר N מספר הפריימים ברצף ו- d מספר המאפיינים לכל מצב, למשל d יכול לכלול 6 מפרקים + מצב גריפר = 7).

לאחר שלב העיבוד, המודל מפיק **פלט**. הפלט תלוי בהגדרת המשימה: בפרויקט נגדיר את הפלט כערך סיווג בינארי המסמן הצלחה/כישלון המשימה, או כערך רציף/קטגוריאלי המסמן את הפעולה הבאה שיש לבצע (למשל פתיחה/סגירה של הגריפר). הפלט יחושב ע"י שכבה אחרונה של Mamba (למשל perceptron לסיווג) על בסיס הייצוג הפנימי לאחר קריאת כל הרצף. ניתן לעצב את המודל כך שיחזיר גם פלט בכל צעד (sequence-to-sequence), אך למטרות פשוטות ניקח במקרה הבסיס פלט סופי אחד לכל רצף.

5.2. ממשקי תוכנה (Software API): בתוך הקוד יוגדר מחלקות ופונקציות עיקריות עם

הממשקים הבאים:

5.2.1 DataLoader - מחלקה/פונקציה לטעינת נתונים: מקבלת נתיב/ים לדאטה,

קוראת את התמונות (באמצעות ספריות כמו OpenCV/PIL) ואת קובצי הנתונים, ומחזירה מבנה נתונים לדוגמה (למשל אובייקט המכיל frames_tensor בגודל $[N, 3, H, W]$ ו-states_tensor בגודל $[N, d]$).

5.2.2 OpenClipEncoder - מחלקה העוטפת את המודל OpenCLIP הטעון מראש.

תהיה לה פונקציה encode(frame_tensor) שמקבלת טנזור תמונה (או אצווה של תמונות) ומחזירה טנזור $[N, 512]$ של embeddings. ייתכן שאני אשתמש ישירות בפונקציה מספריית OpenCLIP ללא המחלקה שלי, אבל ההפרדה עוזרת להבנה: למשל ניתן להשתמש ב-API של הספרייה כגון model.encode_image() ו-open_clip.create_model() לטעינת המודל (בתחילת להמרת תמונה לוקטור. אני אטען את המודל רק פעם אחת בזיכרון) (בתחילת הריצה) כי הוא גדול, ואז אשתמש בו לכל התמונות. כברירת מחדל נשאר את משקלי OpenCLIP קפואים (לא מתעדכנים) כדי לשמור על הידע הטרומ-אימון, אלא אם אחליט בפאזה מתקדמת לבצע Fine-tuning קל.

5.2.3 MambaModel - מחלקה המגדירה את מודל הרצף. אם קיים מימוש זמין

מספריית state-spaces (למשל state-spaces.Mamba מ-GitHub הרשמי), אשתמש בו; אחרת, אגדיר רשת PyTorch ידנית בהתאם למפרט המאמר. הממשק יהיה בדומה ל-RNN פייטורצ'י: פונקציה forward(sequence) שמקבלת סדרת וקטורים (למשל טנזור בגודל $[N, p]$ כאשר $p=512+d$ אחרי שאיחדתי embedding ותכונות רובוט לכל צעד), ומחזירה ϵ_{τ} וקטור פלט בודד (בגודל מספר המחלקות בסיווג, או בגודל ערך גרסיה) או רצף פלטים. בתוך המחלקה ייתכן שישמר גם את המצב הפנימי (Hidden state) של ה-SSM כדי לאפשר עיבוד ממושר, אבל מבחינת הממשק זה שקוף למשתמש. עבור אימון, אגדיר פונקציה train_on(batch) שמקבלת קבוצת רצפים, מריצה אותם דרך המודל,

מחשבת Loss (למשל באמצעות CrossEntropyLoss במידה וזו בעיית סיווג הצלחה/כישלון), ומבצעת עדכון משקלים (של Mamba ואולי של OpenCLIP אם החלטתי להתאים).

5.2.4. RobotSequencePipeline - ניתן להגדיר מחלקת-על או פשוט להשתמש בסקריפט ראשי המשלב את הכל: קורא דאטה, מפעיל את OpenClipEncoder על כל תמונה, מאחד את התכונות עם מצב הרובוט, ומעביר למודל Mamba לקבלת תחזית. ה-Pipeline הזה בעצם מממש את האלגוריתם בשלבים שהוגדר לעיל.

5.2.5. גרסאות וספריות: המערכת מפותחת בשפת Python 3.x בסביבת PyTorch. אני אשתמש בפלטפורמת PyTorch (גרסה עדכנית, למשל 2.0+) עם תמיכת CUDA לצורך האצת חישובים על GPU. ספריית OpenCLIP משמשת לטעינת מודל ה-CLIP המאומן מראש ולחישוב ה-embeddings. ספריית HuggingFace/transformers יכולה לשמש כחלופה לטעינת מודלי CLIP, אך נשתמש ישירות במימוש מ-MLFoundations (הקוד הפתוח של OpenCLIP) לטובת פשטות התאמה לאימון. עבור מודל Mamba, אשתמש כאמור במימוש זמין (אם קיים) או נעזר בספריות מתמטיות (כמו PyTorch Lightning עבור אימון נוח, אולי) כדי להרכיב את המודל.

נקודות אינטגרציה וגרסאות: יש לשים לב להתאמות גרסאות - למשל גרסת CUDA תואמת לגרסת הדרייבר על המכונה (CUDA 11.8+ תואם לכרטיסי NVIDIA מודרניים). PyTorch יסופק עם התאמה ל-CUDA. ספריית OpenCLIP דורשת כנראה Python 3.8 ומעלה. אני אציין בדוח התיכון את גרסאות הספריות המדויקות שאשתמש בהן (לדוגמה: OpenCLIP 2.7.0, PyTorch 2.1 וכד'), ואנהל את התלויות באמצעות Conda למניעת בעיות תאימות.

6. מידע / דאטה

לרשותי עומד דאטהסט מובנה בתחום לימוד הרובוטיקה: פרויקט זה ישתמש ב-BridgeData, בסיס נתונים פומבי שנחשב Benchmark בתחום. BridgeData (גרסה 1 או 2) הוא אוסף גדול ומגוון של הדגמות רובוטיות לביצוע משימות מטבח שונות. למשל, הוא מכיל 7,200 הדגמות של הרובוט המבצע 71 משימות (כמו להזיז חפצים, לפתוח מגירות, לנקות וכו') ב-10 סביבות שונות. הנתונים נאספו עם זרוע רובוטית מסוג WidowX250 (6 דרגות חופש) המצוידת במצלמה, כך שלכל הדגמה יש רצף תמונות מצולם וכן את הטרגטורייית התנועה של הרובוט (רצף המצבים שלו) לאורך זמן. כל הדגמה מתויגת כ"משימה מוצלחת" (הרובוט השלים את המשימה) או ככישלון, מה שיאפשר גם לבחון יכולת חיזוי הצלחה.

6.1. מבנה הנתונים: עבור כל הדגמה (trajectory) בדאטהסט יש:

6.1.1. רצף תמונות: פריימים שצולמו במהלך ביצוע המשימה, בדרך כלל ממצלמה שמסתכלת על זירת העבודה. קצב הצילום יכול להיות כמה FPS בודדים (לפי הקוד המסופק עם הדאטהסט). התמונות יהיו בפורמט RGB, בגודל מקורי (למשל 240×320 times) - אני אבצע שינוי גודל (resize) ו-Normalize בהתאם לדרישות מודל ה-OpenCLIP (בדרך כלל 224×224 times פיקסלים).

6.1.2. רצף מצבי רובוט: לכל פריים קיים וקטור נתוני חיישנים/מצב. הווקטור כולל בין היתר את זוויות המפרקים של הזרוע, מצב הגריפר (פתוח/סגור או ערך רציף של לחץ), ואולי נתוני אינרציה/כוח אם יש (ב-BridgeData ספציפית כנראה מתמקדים במצב מנועים וגריפר). בנוסף, ניתן לגזור מזה את מיקום האנד-אפקטור (קצה הזרוע) במרחב, אך לרוב אתמקד בנתונים הגולמיים.

6.1.3. תווית/ות פלט: בחלק מהמקרים אגדיר לכל רצף תווית הצלחה (1/0). כמו כן, ניתן להגדיר תווית לכל צעד – למשל מה הפעולה שהתבצעה (אך מכיוון שאלו הדגמות ולא פקודות, נשתמש פשוט במידע המצבים).

הנתונים יחסית גדולים בנפח – קובץ ההדגמות כולו עשוי להיות עשרות גיגה-בייט (כ-GB30 רק בגרסת הנתונים הראשונית הורדה למכונה). לכן אצטרך להתחשב בכך בעת אחסון וטעינה. המכונה הווירטואלית כוללת מספיק מקום כדי לאחסן את הנתונים, אך ייתכן שארצה לבצע אגרגציה או דגימה אם זיכרון ה-GPU יהווה מגבלה. למשל, אפשרות אחת היא לחשב מראש את ה-embeddings לכל התמונות בדאטהסט ולשמור אותם לקובץ בינארי, ובכך לחסוך את הצורך להעביר שוב ושוב את כל תמונות הגלם דרך OpenCLIP במהלך האימון. בשלב התיכון, נניח שאוכל להחזיק בדיסק את כלל הנתונים ולטעון כל פעם אצווה (batch) של רצפים לזיכרון לעיבוד.

6.2. ניהול ואחסון: לא נדרשת מערכת אחסון מורכבת – קובצי הנתונים ישמרו במבנה תיקיות סטנדרטי (למשל לכל משימה תיקיה עם תמונות וקובץ נתונים). אשתמש בפונקציות Python לקריאת התמונות (ספריית PIL) ולקריאת קובצי מצבים (ספריית csv/json). כדי להאיץ, אשקול שימוש ב-HDF5 או pickle לטעינה מאוחדת, אך זה לשיקול ביצועי. בנוסף, אוודא סינכרון מדויק לפי חותמות זמן: אם למשל קצב צילום התמונות וקצב לוג הנתונים אינם תואמים בדיוק, אבצע אינטרפולציה או התאמה כדי לשדרך כל תמונה למצב הרובוט המתאים ביותר באותו רגע. למעשה, אתגר סנכרון הנתונים צויין כעניין חשוב – לעיתים ייתכן פער של עשרות מילי-שניות בין צילום פריים לרישום מצב, ולכן אצטרך לטפל בכך בעיבוד המקדים (למשל באמצעות שדה timestamp לכל רשומה).

6.3. אתגרים נוספים בעיבוד הנתונים: מלבד הסנכרון:

6.3.1. רעש וגיוון ויזואלי: תמונות מכילות רעשי רקע, חפצים מפריעים, ותנאי תאורה משתנים. אשתמש ביכולת של CLIP להדגיש features עיקריים ולהתעלם מרוב הרעש, אך אשקול גם data augmentation (כגון שינויים אקראיים בתמונות) בזמן האימון כדי לחזק את עמידות המודל.

6.3.2. רצפים ארוכים מאוד: חלק מהדגמות עשויות לכלול מאות פריימים. כדי ללמד את Mamba בצורה יעילה, ייתכן שאחתוך רצפים ארוכים מאוד למקטעים קצרים יותר (windowing) או אדלל (sampling) את הפריימים (למשל לקחת כל פריים שני/שלישי) – זאת על חשבון פירוט לאורך הזמן. פרמטרים כגון Window Length ו-Frame Offset כבר נשקלו כאופציונליים בממשק, כדי לאפשר ניסוי עם אורך רצף שונה. החלטה זו תתקבל בניסוי מקדים: אתחיל עם רצף מלא ואם נראה שקשה לאמן כך, איישם דגימה.

6.3.3. איחוד פורמט: עליי לייצר ממידע הגלם (תמונות וערכי חיישנים) ייצוג אחיד שהמודל יוכל לעבד. זה אומר: הפיכת כל התמונות לטנזורים נומריים מנורמלים, איחוד יחידות ומידות של נתוני הרובוט (למשל נרמול זוויות למעלות או רדיאנים עקביים, קידוד מצב גריפר כ-0/1), ולבסוף צירוף וקטור הנתונים לווקטור ה-embedding כך שנקבל וקטור מאוחד לכל צעד. את התוצאה הזו (רצף וקטורים מאוחדים) ניתן להזין ישירות ל-Mamba.

7. תיאור הגישה לפתרון

הרעיון הכללי: הגישה שלי לפתרון היא לחלק את בעיית "למידת הרצף החזותי" לשני תתי-בעיות: **(1) למידה חזותית סטטית** – כיצד לייצג כל תמונה בצורה מיטבית, ו-**(2) למידה דינמית** – כיצד ללמוד מהתמונות הללו את הדפוס לאורך הזמן. עבור **חלק 1**, אשתמש כאמור במודל OpenCLIP כ-Encoder חזותי. OpenCLIP אומן על כמויות עצומות של נתוני תמונה-טקסט ולכן פיתח הבנה פנימית של תוכן תמונה (עצמים,

פעולות, הקשרים). הוא מסוגל לקחת תמונה חדשה (שלא נראתה באימון המקורי) ולהפיק עבורה embedding – מעין תקציר של המאפיינים החשובים בה – באופן עקבי. משמעות הדבר היא שאם שתי תמונות שונות מכילות תוכן דומה, הווקטורים ש-OpenCLIP יחזיר עבורן יהיו קרובים במרחב, ולהפך. תכונה זו מועילה כדי להכליל על וריאציות בתמונות (למשל זווית ראייה שונה של אותה סצנה תיתן embedding דומה). בנוסף, המעבר לתיאור וקטורי קצר מאפשר לעבד רצפים ארוכים ללא עומס חישובי כבד – במקום לעבד מאות תמונות מלאות בכל צעד, נעבד מאות וקטורים (512 מספרים כל אחד), מה שקטן משמעותית בעלות.

בחלק 2, לאחר שיש רצף של תיאורים חזותיים (וגם מידע מצב הרובוט לכל צעד), המטרה היא ללמוד את הקשר הטמפורלי – איך מצב המערכת מתפתח מפריים לפריים. כאן נכנס מודל Mamba, שהינו ארכיטקטורה חדשנית ללמידת סדרות זמן ארוכות. Mamba שייך למשפחת SSM (State Space Models), שבהם הרעיון הוא לשמר מצב חבוי שמתעדכן בכל צעד זמן, וכך "לזכור" מידע מעבר רחוק תוך שמירה על חישוב יעיל. בעזרת Mamba, המערכת יכולה לקלוט רצף ארוך (נאמר +100 צעדים) וללמוד תלות ארוכת טווח – למשל, להבין שמה שהרובוט ראה ועשה בתחילת הרצף משפיע על הצלחתו בסוף הרצף. אחד היתרונות הגדולים של Mamba הוא היכולת שלו להתמודד עם רצפים ארוכים בקירוב ליניארי – כלומר, אפילו אם נוסף עוד ועוד פריימים, זמן החישוב גדל בצורה פרופורציונלית (ולא ריבועית כמו ב-Transformer). כך נוכל לספק למודל כמה שיותר היסטוריה ללא חשש שנחנק מחישובים.

למה דווקא שילוב זה? כפי שפורט בהחלטות העיצוב, השילוב OpenCLIP+Mamba נותן מענה משולב לכל הדרישות: OpenCLIP מטפל במורכבות המידע הוויזואלי ומייצר ייצוג עשיר אך קומפקטי לכל תמונה, ו-Mamba מטפל בממד הזמן ומסוגל לקשר בין אותם ייצוגים על פני רצף ארוך בצורה יעילה. שילוב נתוני הרובוט מתבצע בצורה טבעית ע"י הדבקת הווקטורים, ומכיוון שמדובר במידע נומרי רציף (לדוגמה זוויות מפרקים), Mamba מסוגל למזג מידע מולטימודלי – למעשה עבודת מחקר עדכנית (FlowRAM) השתמשה במודל Mamba כדי למזג מידע תלת-ממדי עם מידע חזותי לשם שליטת רובוט, והראתה שמודל זה יעיל במיזוג מולטימודלי ומביא לשיפור ביצועים עם עלות חישובית קטנה. יתר על כן, Mamba תוכנן באופן מודולרי כך שניתן לשלבה כחלק ממערכת גדולה: למשל, ב-FlowRAM Mamba משמשת כמודול Fusion נפרד. גם אצלי, Mamba פועל כרכיב שניתן לאימון עצמאי על גבי התכונות הוויזואליות, מה שמיישם את עקרון Separations of Concerns שתורם ליציבות הפתרון.

השוואה מול גישות חלופיות: אם הייתי מנסה לגשת לבעיה בגישה מסורתית יותר, לדוגמה לאמן רשת אחת גדולה שקולטת פיקסלים ומוציאה החלטה, סביר להניח שהייתי נתקלת בקשיים. רשת כזו (בין אם CNN+LSTM או Transformer end-to-end) הייתה דורשת כמות עצומה של נתוני אימון כדי ללמוד מאפס הן את עיבוד התמונה והן את דינמיקת הרצף. לעומת זאת, הגישה הזו מנצלת למידה מועברת OpenCLIP – (Transfer Learning) מביא "ידע חזותי מוקדם" שנלמד ממקור אחר, ואני צריכה רק לכוונן/לנצל אותו, בעוד Mamba מביא "יכולת זיכרון יעילה" שנבדקה כבר על משימות רצף כלליות. במילים אחרות, אני בונה על כתפיהם של מודלים חזקים במקום להתחיל מאפס. גישה חלופית נפוצה ברובוטיקה היא שימוש ב-LSTM עם תכונות ידניות – אולם כפי שצינתי, LSTM יתקשה ברצפים ארוכים מאוד. גם Transformer מודרני דוגמת מודל RT-1 (של גוגל) דורש משאבי עתק, בעוד שאני שואפת לפתרון רזה יותר שיוכל אולי אף לרוץ בזמן אמת על חומרה מוגבלת.

לסיכום, הגישה שלי משלבת Encoder חזותי כללי ו-Decoder רצף ייעודי, מתוך אמונה שהפרדה זו מתקסם את ניצול הנתונים והמודלים: נקבל תובנות מכל תמונה ללא אובדן מידע קריטי, ונלמד מהתזמון והסדר את המשמעות לפעולות הרובוט.

8. פסאודו-קוד / הסבר עיוני של הפתרון

אציג כעת תיאור בשלבים של פעולת המערכת (פסאודו-קוד כללי), כדי להבהיר כיצד המרכיבים משתלבים בפעולה רציפה:

בתחילת תהליך האימון, אטען את מודל OpenCLIP המאומן מראש במצב הערכה (או אימון, אם נבצע, fine-tuning) ואת מודל Mamba שאותו חל עם משקלים רנדומליים. אגדיר גם אופטימיזר (למשל Adam) לצורך עדכון המשקלים של Mamba (ובמקרה הצורך גם של שכבת הפלט או חלק מ- OpenCLIP).

בשלב האימון, אעבור על כל אפיזודה (trajectory) בדאטהסט. לכל רצף כזה יש רצף של תמונות ונתוני מצב. עבור כל תמונה בפריים I_t , נפעיל את OpenCLIP ואחשב את embedding הוויזואלי שלה, כך שנקבל וקטור E_t בגודל קבוע (למשל 512). תהליך זה יתבצע בדרך כלל בצורה וקטורית על כל הפריימים יחד ועל גבי GPU.

לאחר מכן, לכל שלב t אחבר את ה-embedding עם נתוני מצב הרובוט המתאימים (כגון מצב המפרקים או הגריפר) ואצור וקטור משולב $X_t = [E_t; S_t]$ כך מתקבל רצף של וקטורים המייצגים את כל המידע על הפריימים באותו רגע.

את הרצף הזה אצין למודל Mamba. המודל יעבד את סדרת הווקטורים ויחזיר פלט Y שיכול להיות תחזית אחת (למשל האם המשימה הצליחה) או רצף תחזיות (למשל הפעולה שיש לבצע בכל שלב).

כדי לאמן את המודל, אשווה בין הפלט Y לבין תוצאות האמת שמגיעות מהדאטה (למשל רצף פעולות שביצע הרובוט בפועל), ואחשב פונקציית עלות מתאימה – כמו MSE או CrossEntropy לאחר חישוב ה- loss אבצע שלב Backpropagation עם אפס גרדיאנטים, נגזרות לאחור ועדכון משקלים בעזרת האופטימיזר.

בתום האימון, אריץ את המודל על סט בדיקה כדי להפיק תחזיות מבלי לעדכן משקלים. אשווה את התחזיות לתשובות האמיתיות ונחשב מדדים כמו דיוק, שגיאה ממוצעת, או גרפים של ביצועים לאורך זמן. כך אוכל להעריך עד כמה המודל למד את הקשר בין רצף התמונות לפעולות הרובוט, ולשפר אותו בהתאם.

9. כלים וסביבות פיתוח

הפיתוח וההרצה של הפרויקט ייעשו בסביבה המתאימה לחישובים כבדים (למידת עמוקה). דהיינו מכונה עם GPU ותמיכת CUDA. לצורך כך הוקצה לי מכונה וירטואלית (VM) עם כרטיס גרפי ייעודי. התקנתי בסביבה זו את כלי הפיתוח הדרושים: סביבת פייתון מנוהלת באמצעות Miniconda, כולל התקנת ספריות מדעיות נדרשות. כמו כן הותקן VS Code בסביבה לשם עריכת קוד נוחה מרחוק. התקנתי את פלטפורמת PyTorch בגרסה התומכת ב-CUDA (למשל באמצעות הפקודה `conda install pytorch cudatoolkit=11.x -c` (pytorch). כדי לוודא שניתן יהיה לנצל את הכרטיס הגרפי לאימון המודל. במהלך ההתקנה שמתי לב שלא לעדכן/לשדרג את דרייבר הכרטיס במכונה (הנחיה מיוחדת של תמיכת ה-IT) כדי למנוע אובדן תקשורת עם ה-GPU. בנוסף, הדאטהסט (כ-GB30) הועלה למכונה הווירטואלית ואוחסן עליה באופן קבוע – אין מגנון ניקוי אוטומטי, כך שהנתונים יישארו זמינים בין הרצות.

מערכת ההפעלה על ה-VM היא לינוקס 64 (Ubuntu 22.04)-ביט, עם הדרייברים המתאימים ל-NVIDIA GPU. נעשה שימוש ב-shell של אובונטו לעבודה מול conda. לצורך נוחות הפיתוח, התחברות למכונה מתבצעת דרך פרוטוקול Remote Desktop Horizon Client עם אימות דו-שלבי כפי שסופק ע"י המוסד, ולאחר מכן פתיחת VS Code בחיבור מרוחק לעריכת הקוד.

ניהול קוד וגרסאות: את קוד הפרויקט אנהל באמצעות מערכת גיט (Git) – ריפוזיטורי פרטי יכיל את כל קבצי הקוד, ויאפשר מעקב שינויים וגיבוי. ספריות צד-שלישי (open_clip, וכו') ינוהלו דרך conda/pip requirements, ולא אשמור את קודן ישירות אלא רק את רשימת התלויות.

ספריות וכלים עיקריים בפרויקט:

- Python 3.10 – שפת התכנות העיקרית. נבחרה בשל התמיכה הענפה בלמידה עמוקה ובכלים הנלווים.
 - PyTorch 2.x – פריימוורק ללמידה עמוקה שעליה אבסס את המימוש של מודל Mamba ואימון הרשת. PyTorch מספק ממשקי AutoGrad לחישוב נגזרות, אופטימיזציות, וכלים לעבודה על GPU בפשטות.
 - OpenCLIP – ספריית קוד פתוח (מבית MLFoundations) המאפשרת טעינה ושימוש במודלי CLIP מאומנים. אשתמש בה כדי לקבל את מודל ה-ViT-L/14 של CLIP ולהפעילו על התמונות. ספרייה זו גם מתממשת עם PyTorch ולכן שילובה במערכת חלק.
 - Hugging Face Transformers – ייתכן שאשתמש בה עבור כלים נלווים. היא גם חלופה לטעינת CLIP, כאמור.
 - ספריית state-spaces/mamba – אם תעמוד לרשותי (קוד רשמי של Mamba מ-2023), נשלב אותה. במידה ולא, אשתמש בספריות NumPy ו-SciPy לצד PyTorch כדי לממש ידנית רכיבים רציפים הדרושים ל-SSM.
 - אחרים: OpenCV/PIL לקריאת ועיבוד תמונות, Pandas/NumPy לקריאת קובצי נתוני הרובוט, Matplotlib או Seaborn ליצירת גרפים בזמן ניתוח התוצאות.
 - Streamlit 1.x – ספרייה לבניית יישומי ווב פשוטים בפייתון. אשתמש בה כדי ליצור לוח מחוונים (dashboard) להצגת תוצאות הניסויים. הממשק יאפשר לבחור רצף ולראות למשל את ניתוח המסלול הרובוטי (נצייר גרף מצב גריפר לאורך זמן עם ציון נקודות שאותן חזינו). Streamlit נוח כי הוא מאפשר להריץ את כל שלבי ה-Pipeline מרגע טעינת הנתונים ועד הצגת התוצאות בלחיצת כפתור אחת. אקרא לכלי זה "Robotic Trajectory Analyzer".
- לסיכום סעיף זה, סביבת הפיתוח מוכנה לעבודה: יש לנו מכונת GPU עם כל התלויות הדרושות (Python, PyTorch, CUDA וכד'), כלי פיתוח מודרניים (VS Code) ותוכנית להריץ ולבחון את המודל גם בצורה אינטראקטיבית (דרך Streamlit). בכך אני ערוכה לממש את תכן המערכת, לאמן את המודל על הנתונים, ולבסוף להעריך ולהדגים את הפתרון בצורה יעילה וברורה.